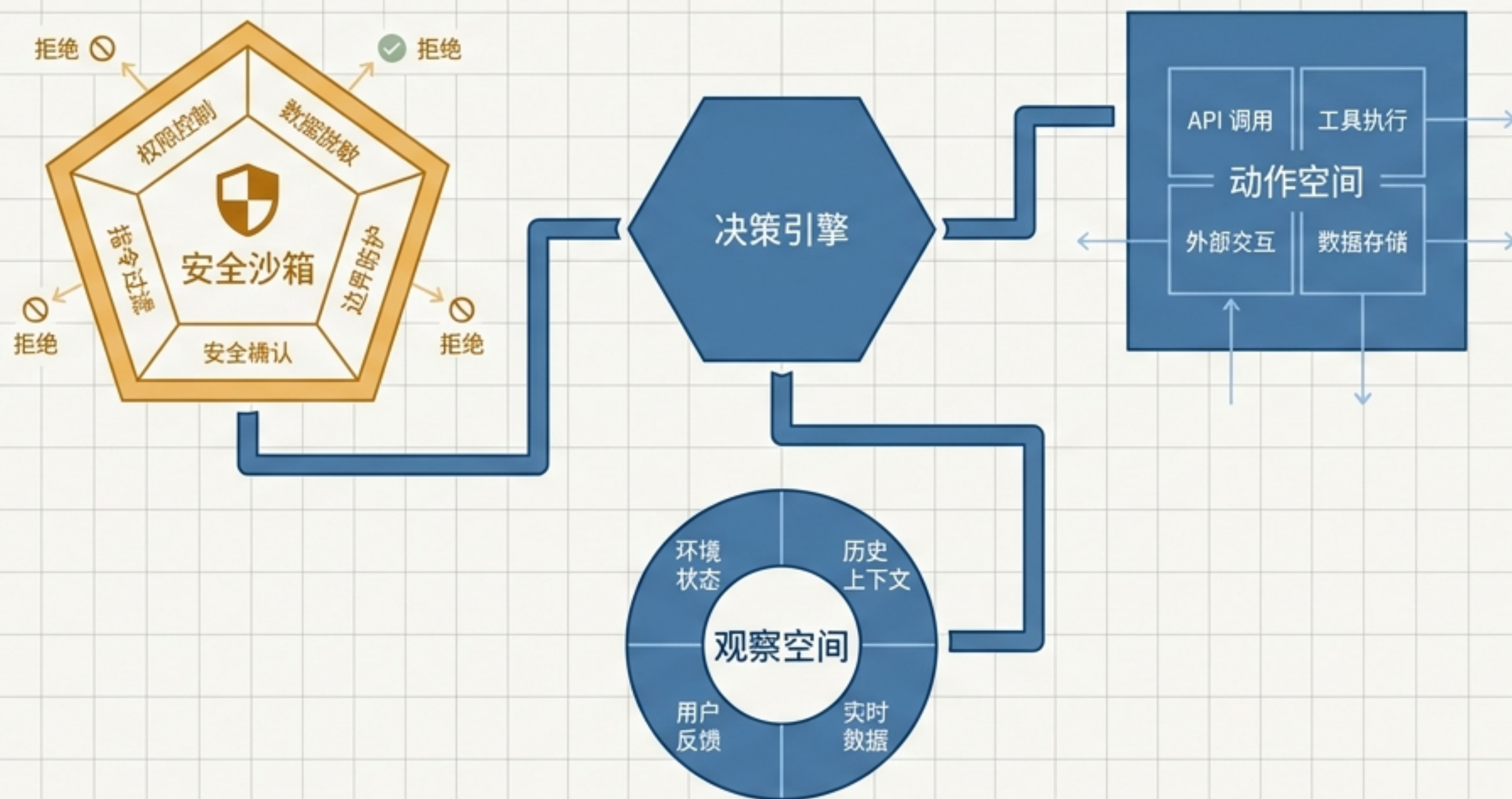


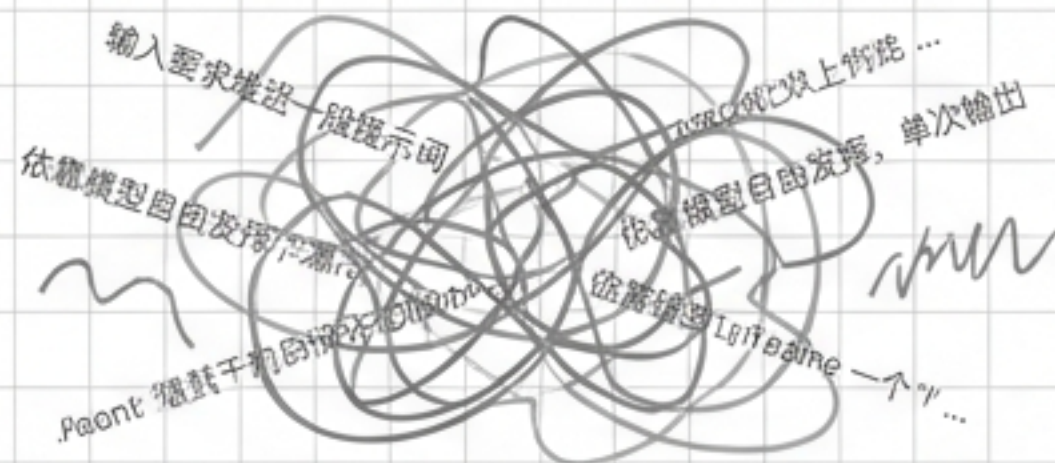
# 从 Prompt 魔法到 Agent 工程

构建可复用、可验证、边界受控的生产级 Agent 应用骨架



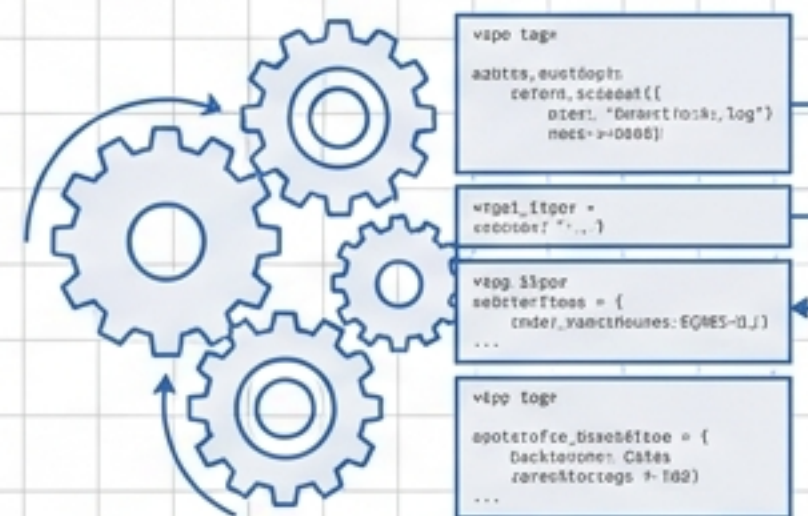
# 认知重塑：告别“神奇的 Prompt”

## 旧心智（智能补全 / Prompt）



输入：所有要求堆进一段提示词  
执行：依靠模型自由发挥，单次输出  
评估：看模型回答得漂不漂亮  
结果：极其不稳定，不可测试，不可交接

## 新心智（Agent 工程化）

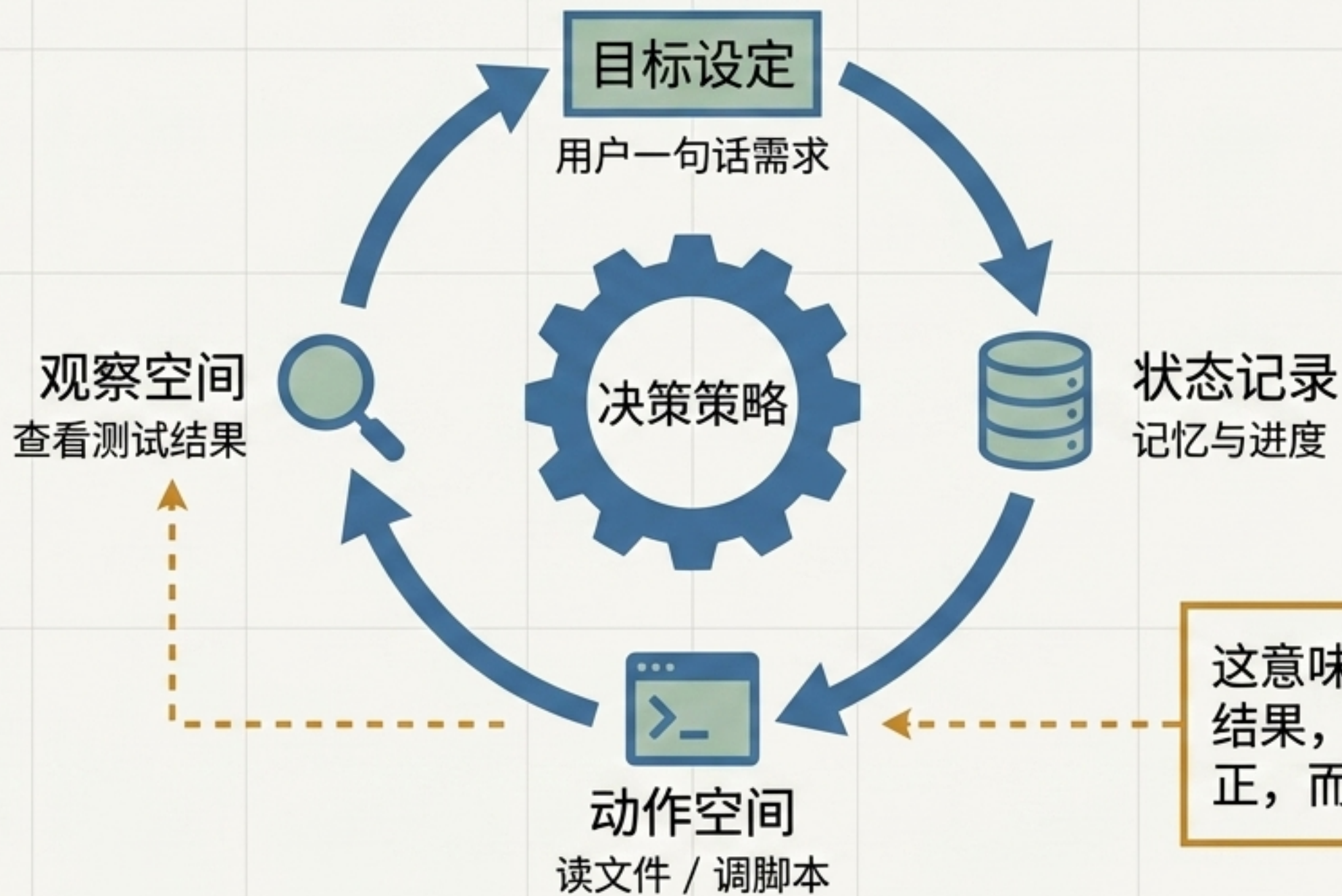


架构：上下文分层外部化（Context Layering）  
设计：核心在于动作空间与安全边界设计  
闭环：强调证据提取、自动验证与持续纠偏  
结果：可复用、可测试的生产级应用骨架

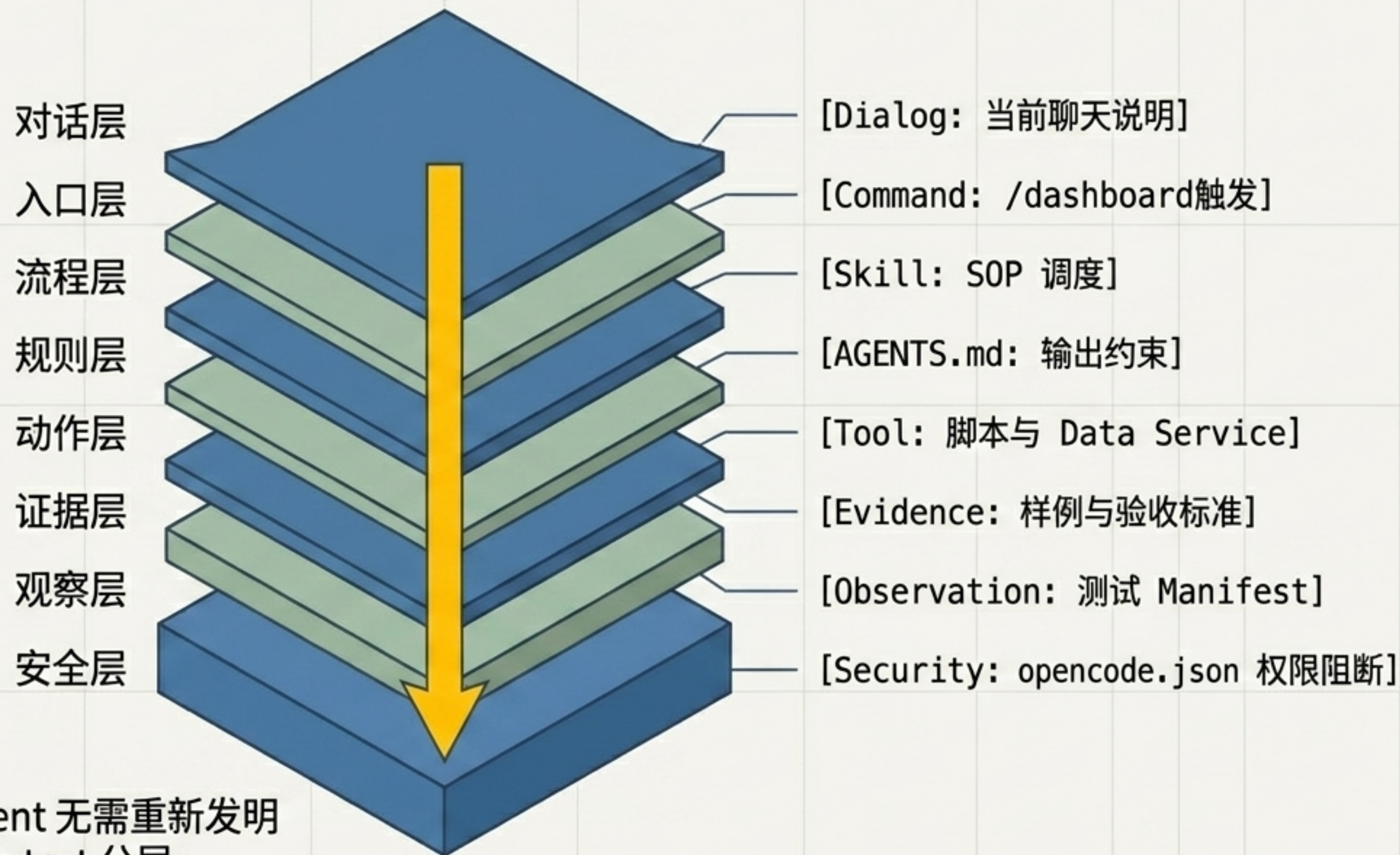
今天的核心议题：如何把业务场景，沉淀成可复用、可交接的 Agent 应用。

## 重新定义 Agent：带状状态的决策循环

“Agent 不是一个大模型，而是一个带状状态的决策循环。”



# 破局之道：OpenCode 上下文分层系统



当用户提出类似需求时，Agent 无需重新发明流程，只需复用已沉淀的 Context 分层。

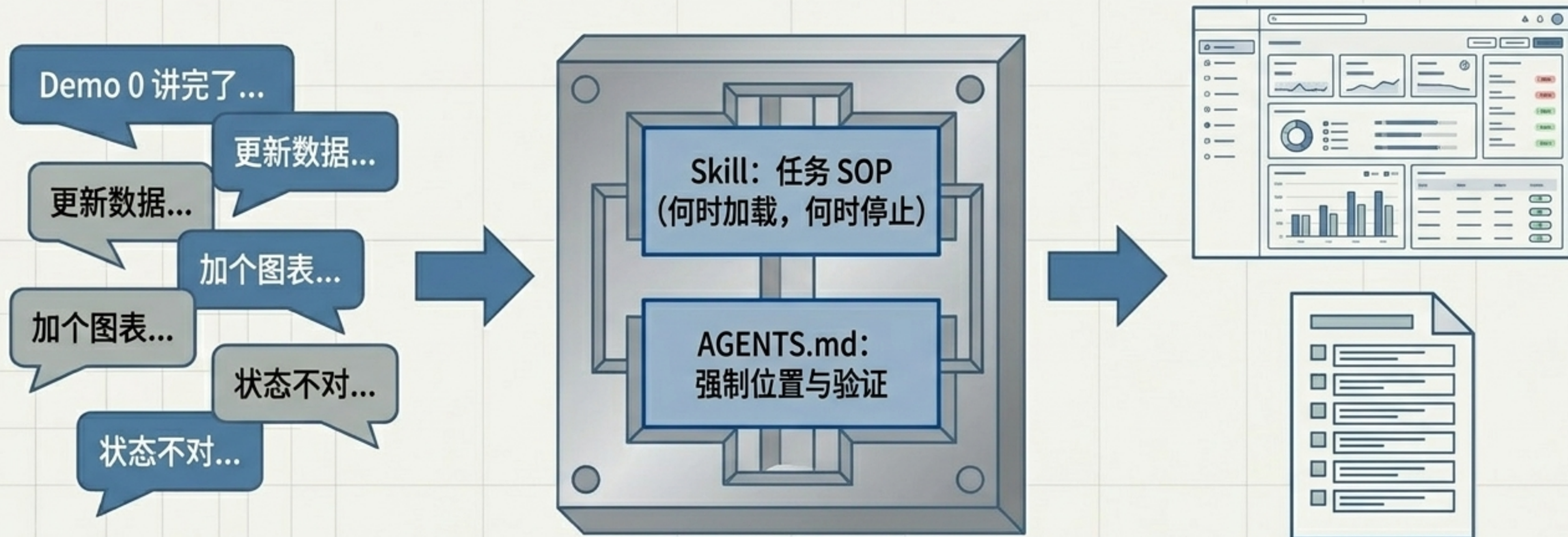
# 业务场景的最小可复用骨架地图



注：这 4 个 Demo 并非具体的复杂业务系统，而是覆盖绝大多数生产痛点的基础架构骨架。

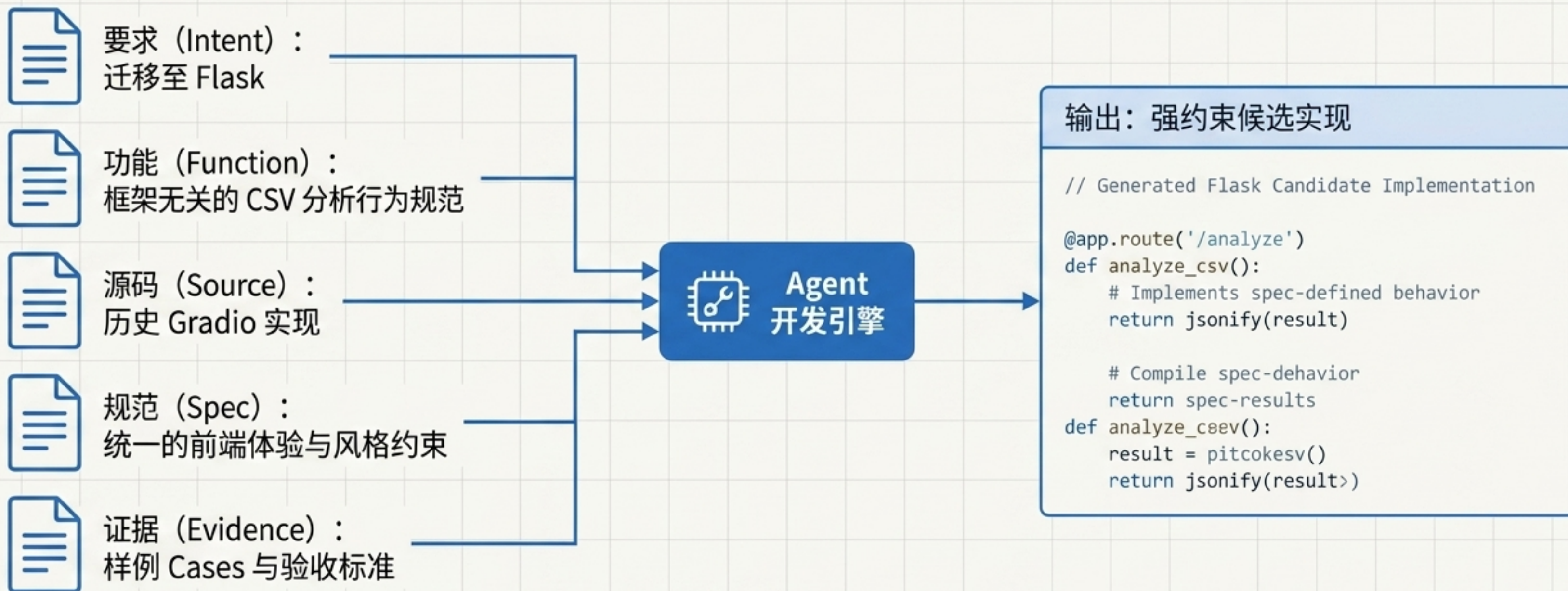
# 场景 00: 规则驱动的 Dashboard 生成

业务痛点: 只给一句零散更新, 模型临时写 HTML 导致字段、样式、状态口径疯狂漂移。



不要让模型自由发挥。核心不是让模型写得更花, 而是把模板、字段、脚本和验证彻底固化。

# 场景 01：文档规范与 Web App 跨框架迁移



复杂的开发迁移不能单纯依靠翻译语法，必须拆分功能、风格与验收证据，独立定义。

# 验证的边界：自动验证通过 $\neq$ 生产安全

## ✓ 自动验证覆盖 (Auto Verification)

CSV 行列与缺失值统计

推荐绘图列逻辑

空数据错误路径覆盖

静态代码风格检查

## ⚠ 必须 Owner Review (Human-in-the-Loop)

生产环境上传安全

超大文件处理性能

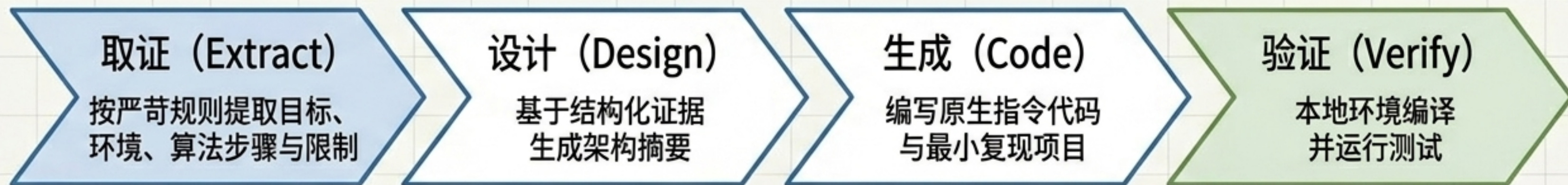
部署方式与混合编码

复杂图表业务需求

警惕过度声称！静态结构和边界行为符合预期，不代表生产安全需求已完全解决。

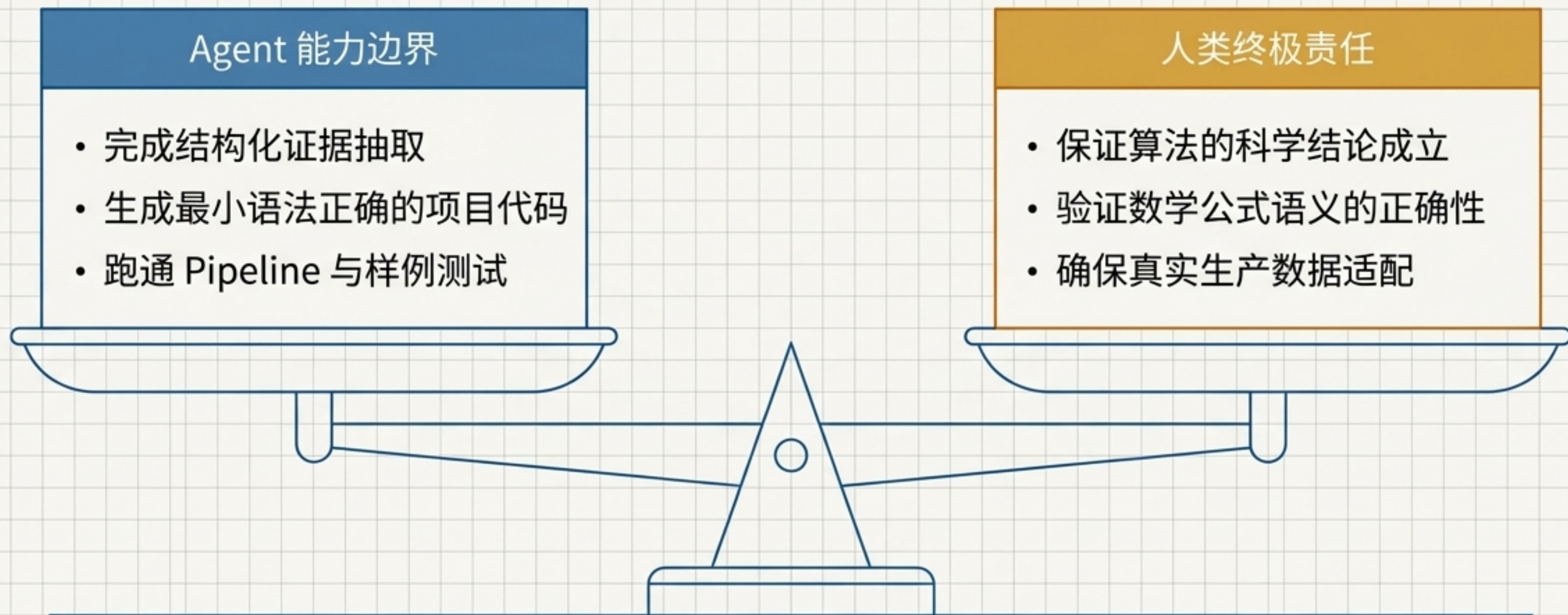
## 场景 02: PDF 信息抽取与复现项目生成

~~让 AI 读论文，然后直接给我代码建议。~~



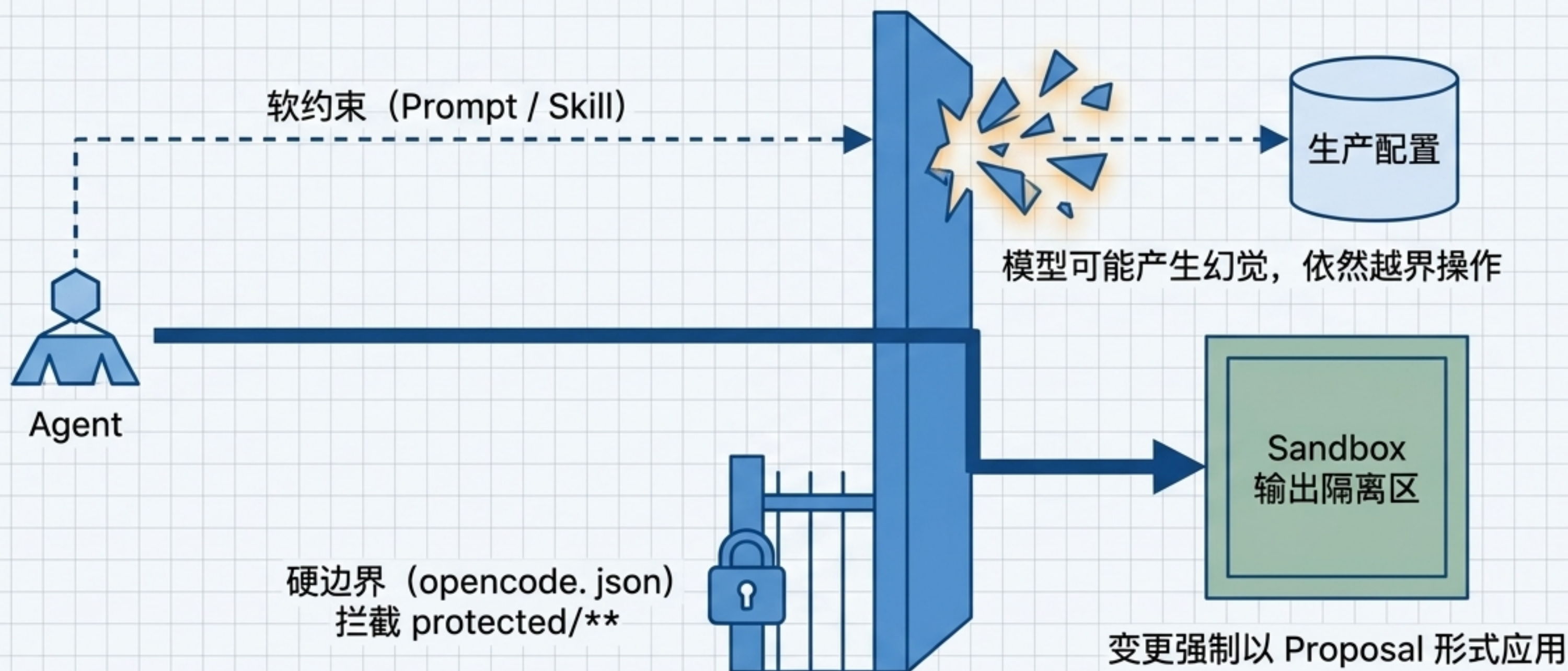
Agent 执行代码动作必须被结构化的 Evidence 和本地验证回路双重约束。

# 人机协同的黄金法则



验证脚本通过，仅仅说明跑通了工程 Pipeline。业务口径、科学真相与语义正确，永远需要人类确认。

## 场景 03-A：权限沙箱与配置变更边界



安全不能依靠模型的“道德约束”，必须依靠底层运行时的物理拦截。

## 场景 03-B：受控的数据服务获取

### The Wrong Way



Agent



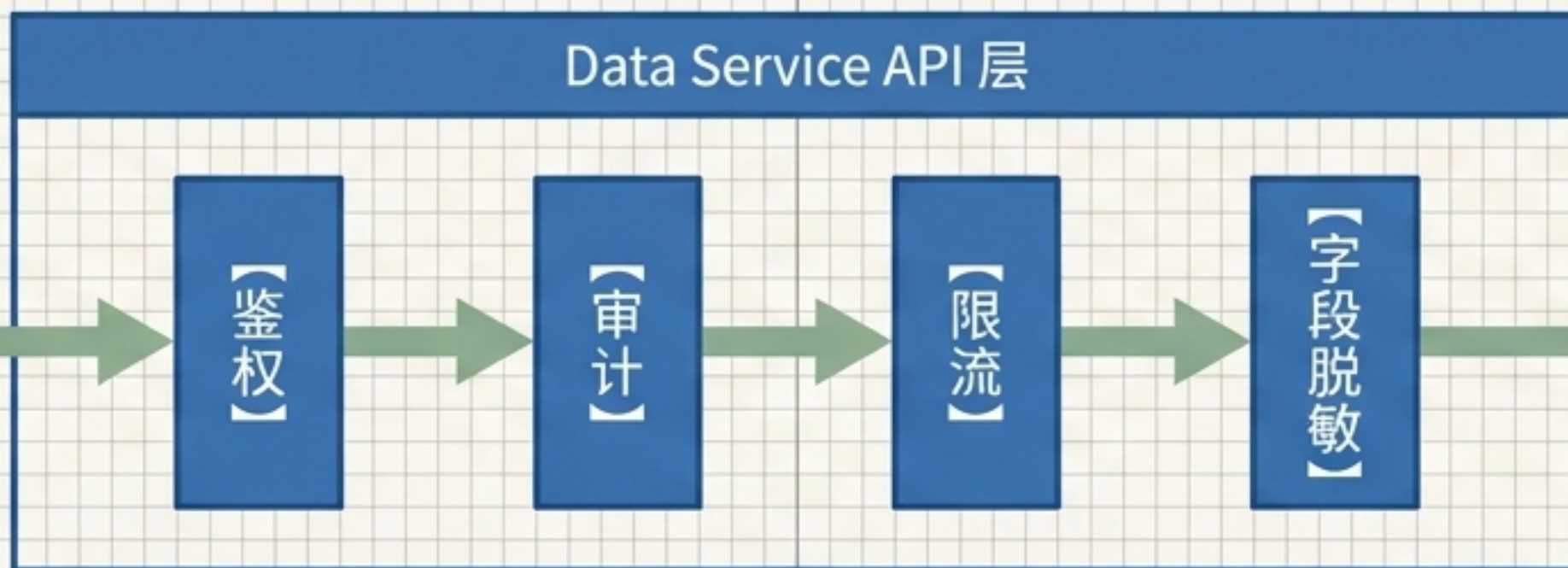
直接连 DB：数据泄露风险、全表扫描  
引发灾难、无差别读取原始字段。



### The Right Way



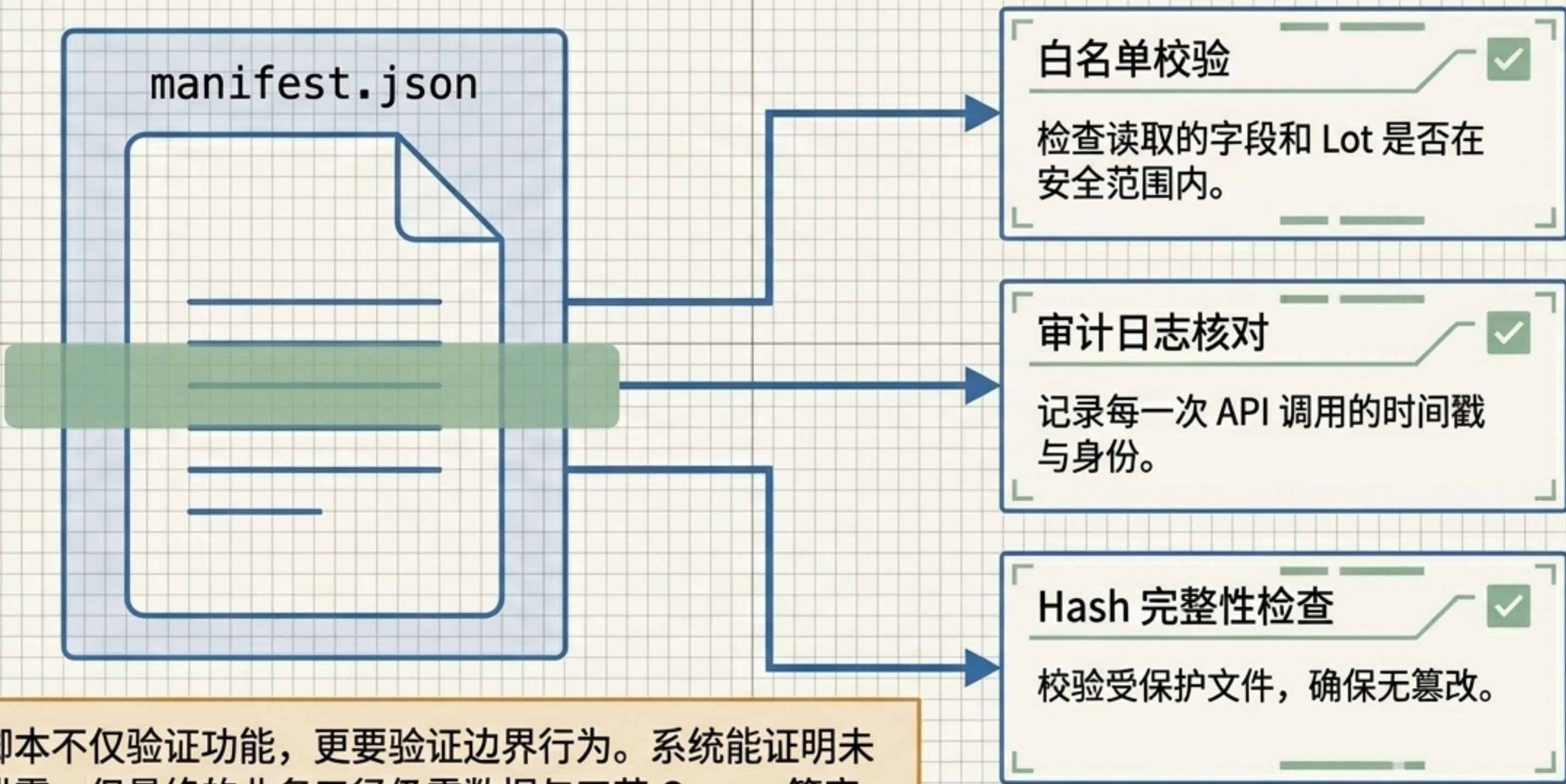
Agent



仅返回聚合后的结果

生产里的 Agent 不是权限全开的自动化脚本，而是被封装在受控动作空间里的协作者。

# 验证数据的“法医学”检查 (Validation Manifest)



验证脚本不仅验证功能，更要验证边界行为。系统能证明未发生泄露，但最终的业务口径仍需数据与工艺 Owner 签字。

# 核心沉淀：Agent 在生产中站稳脚跟的四大方法论

## Production-Grade Agent

### 先动作，再自治

没有工具约束、权限边界和验证回路的 Agent，只是在成倍放大不稳定性。

### 先取证，再生成

必须先建立取证、验证与纠偏机制，再要求自动完成。

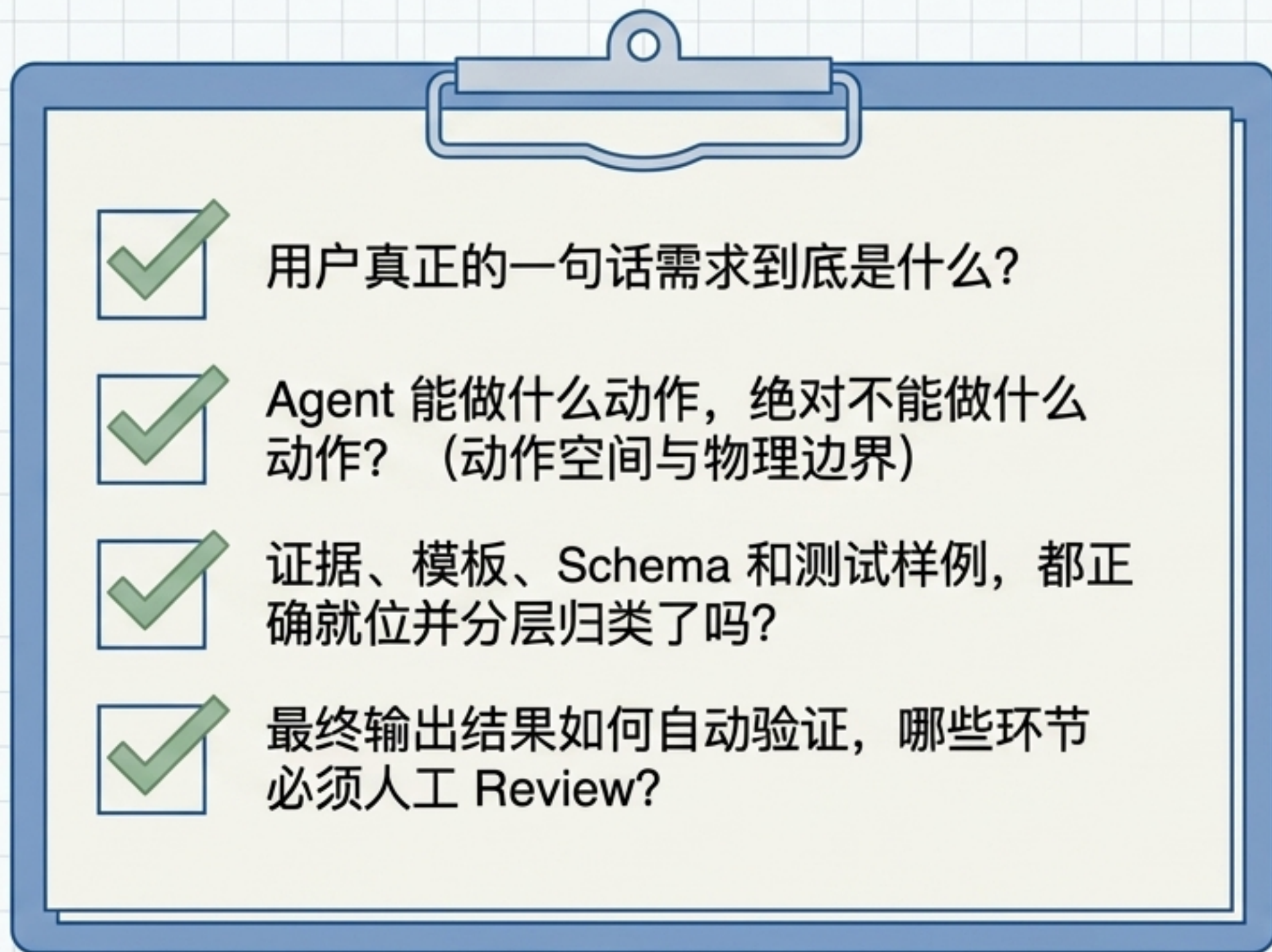
### 重系统，轻提示词

Context 必须分层，它是一个可交接的系统，绝不是堆砌的 Prompt。

### 全生态配合

运行时 + 记忆 + 协议 + 工具 + 安全 + 工作流的全面协同。

# 你的 Agent 起飞前自检清单 (Pre-flight Checklist)



> 构建 Agent 的重点不再是魔法，而是严谨的工程设计。